

ForgeData™

Technical Brief

VERSION 1.2.6 · OCTOBER 2026

Forged For Speed. Refined by Craft.

SEAGLASS FOUNDRY LLC

The data-intelligence layer of the Forge suite: ask for data by intent and get back the best-fit correct dataset — ranked by coverage and resolution, with access cost as the deciding tiebreaker.

Executive Summary

A modern geospatial workload almost never lives in one place. A single project pulls elevation from an SRTM mirror or the USGS 3DEP service, imagery from a Sentinel-2 STAC catalog on S3, hydrography from an internal PostGIS, building footprints from an ArcGIS REST service, bathymetry from NOAA, point clouds from local LAZ files, and pre-computed slope rasters from last week's pipeline run. An analyst in QGIS handles that by knowing which file to open. An automated system cannot.

ForgeData™ is the component of the Forge suite that answers, programmatically, on every routing request: which datasets cover this area, which are at the right resolution for what is about to be computed, which is cheapest to read, and whether the suite has already computed the result. ForgeGIS™ computes. ForgeMind™ reasons. ForgeGIS Studio™ and Seaglass Globe™ are where people see the answers. ForgeData decides which data to use and how to get it — solved once, in one place.

14	**1 file**	**5**	**0**
source types behind one query interface — files, catalogs, databases, OGC services and public APIs	the whole catalog and the recipes that refresh it, in one GeoPackage with no secrets inside	access tiers, cheapest first — and remote costs that calibrate themselves on measured latency	guesses: a coverage gap returns a structured reason and the source that could fill it, never the nearest wrong tile

What you would otherwise build yourself

A team that adopts a raw stack — GDAL, PostGIS, a STAC index, some cloud COGs — and wires an agent or a pipeline on top ends up hand-rolling four things. ForgeData is those four things, built once:

1. **Configuration that cannot drift from the data it describes.** The catalog and the recipes that refresh it live in one GeoPackage file — no companion `sources.yml` to fall out of sync, no secrets baked in. (*The catalog*)
2. **A binding from operations to their data requirements.** A registry declares, for each spatial operation, the category, resolution ceiling and band count it needs, so routing is a lookup rather than per-operation conditionals. (*The operation registry*)
3. **Fidelity-first routing across heterogeneous sources.** Candidates that cover the area adequately are ranked finest-resolution first, then by extent-fit, with access cost as the deciding tiebreaker, then deduplicated to a minimal covering set. (*Cost-aware routing*)
4. **Memory of what was already computed.** Derived products register back into the catalog and are preferred over recomputation. (*Derived products and publishing*)

Routing degrades gracefully with sparse metadata rather than failing. Coverage and resolution are the primary sort keys, so thin metadata yields a coarser ranking, not an empty result: unknown-resolution rows are kept unless a caller sets a hard fidelity floor, imagery of unknown band count passes the band filter, and per-band single-band imagery still routes. Richer metadata sharpens the ranking; it is not a precondition for an answer.

What is new since the first release

- **1.2.0** — the **USGS 3DEP** source type (keyless 10 m elevation over the United States); `surface="bathymetry"` and a marine `surface="auto"` fallback for elevation queries; LiDAR rasterised to DSM, DTM, intensity or density GeoTIFFs; an ocean-acoustics operation. The licence was defined as proprietary.
- **1.2.3** — persistence for the ForgeGIS digital twin's ledger, and a twin-recording category.
- **1.2.4** — `render_map` (a hillshade image with a vector overlay); elevation at many points and zonal statistics inside a polygon; a temporal catalog with date filters on search and routing; a static STAC export; CLI commands that show data; storage clean-up; local COPC point-cloud reads; embedded COG overviews; and a round of correctness fixes.
- **1.2.5** — a **shared, read-only machine catalog** beside each user's own; Seaglass Globe scenarios registered by reference; OSM roads found by name; bounded replies (an inline-geometry cap and default paging) with a sandboxed export path for full results; a `catalog_read_only` error; a selectable journal mode for shared catalogs.
- **1.2.6** — a testing, validation and correctness release. Elevation answers state their **vertical units, datum and where those came from**; every reply with no data names an `empty_reason` and every refusal carries an `error_code`; region statistics report the share of the area they actually describe (`coverage_fraction`); every distance, length and area is measured on the **WGS84 ellipsoid**. Projected DEMs and LiDAR clouds in feet now read correctly, and core queries are checked against GDAL. The MCP surface is pinned by a versioned contract, at **2.0.0**: its one breaking change is that `read_raster_window` returns the file's real data type. The runnable jar is now `forgedata-1.2.6-all.jar`, and the PostgreSQL JDBC driver moves to 42.7.13 for two security fixes.

Where ForgeData Sits in the Suite

The routing work could live inside ForgeGIS, ForgeMind or Studio. It deliberately does not: each has a sharp job, and folding data concerns into any of them would blur that boundary.

Component	Its job	Why routing does not belong there
ForgeGIS	GPU geospatial compute — raster I/O, terrain, spectral indices, point clouds	A compute library should stay focused on pixels and geometry; a catalog and sync scheduler would stop it being embeddable
ForgeMind	The agent — turns a request into a validated pipeline	If routing lived here, every other consumer would go through the agent or reinvent the catalog
ForgeGIS Studio · Seaglass Globe	Where operators see and run the suite	Their data screens are thin fronts over the same catalog; storage inside a UI couples browser concerns to data
ForgeData	Data intelligence — catalog, cost routing, sync, derived-product memory, window extraction	The home for the cross-cutting problem, able to evolve independently of every consumer

How the pieces connect. ForgeMind runs ForgeData as a Model Context Protocol server over stdio JSON-RPC — the same surface any third-party MCP client can use — and Seaglass Globe runs its own copy the same way. ForgeGIS Studio embeds ForgeData in-process as a Java library. All of them read the same catalogs. ForgeData in turn consumes ForgeGIS as an in-process library for the actual data reads, so windowed extraction and elevation queries happen with no network hop. The underlying sources stay where they are; no customer has to move data into a new store.

The Catalog: Configuration as Data

The unit of state is a single `.gpkg` file — a GeoPackage, which is SQLite with the standard GeoPackage metadata tables — plus two tables of ForgeData's own. There is no separate inventory file, manifest or lock file. Copy the file to another machine and ForgeData there sees the same catalog; relative paths resolve against the catalog's configured data root.

Table	One row per	Read by	Written by
`datasets`	Dataset you can query — a tile, a layer, a derived raster	Every query and routing tool	Ingesters, dataset insert, derived-product registration, publishing
`data_sources`	Place ForgeData fetches from — a directory, an API, a database	Sync and preview, the auto-sync scheduler	`add-source` (CLI) / `add_data_source` (MCP)

The catalog *is* the configuration: data-source recipes live in the same file as the inventory, so there is nothing to drift. A single sync typically inserts many dataset rows for one source row.

> **Why a catalog is safe to ship.** API keys never live in the catalog. It stores only the name of the environment variable that holds a key or password — never the secret. In the Forge suite the keys themselves sit in the operating system's credential store and are handed to ForgeData as environment variables when it starts. That is what makes a `.gpkg` safe to copy, send to a field laptop or disconnected site, or commit to an internal repository.

The dataset row. A handful of fields drive every decision downstream: `category` (ELEVATION, IMAGERY, BATHYMETRY, VECTOR, LIDAR, the derived VISUALIZATION and ANALYTICAL, and since 1.2.x TWIN_RECORDING and SCENARIO among others); `format` (GeoTIFF, COG, SRTM HGT, NetCDF, JPEG2000, Shapefile, GeoJSON, LAZ and more); four indexed bbox columns; `file_path`, whose scheme sets the access tier (a bare path or `managed://` is local; `postgis://`, `arcgis://`, `http(s)://`, `s3://`, `wcs://`, `wfs://` are remote); `resolution_m`; `band_count`; and, since 1.2.4, start and end datetimes.

Integrity and concurrency. Each MCP call is its own transaction; there is no multi-call atomicity. Catalogs open in SQLite WAL mode with a busy timeout by default, so several processes on one host share a file — concurrent readers, one writer, a second writer waiting rather than failing. WAL is local-disk only. Since 1.2.5 `FORGEDATA_JOURNAL_MODE` selects WAL or DELETE, and a read-only catalog sets no journal mode at all. A schema migration runs on every writable open and is forward-only; a catalog opened read-only is never migrated by that open, and one the account

cannot write must be prepared by the `init-catalog` command, which the suite installer runs. A partial unique index on the content hash defends against duplicate rows even across two JVMs writing the same file.

Federated catalogs and the shared machine catalog

A **catalog set** federates several catalogs: reads fan out across every one and merge with namespaced IDs; writes go to the primary. Since 1.2.5, secondaries are always opened read-only and their source recipes are ignored, and writing to one returns `catalog_read_only`.

The Seaglass Control Center™ uses exactly this: each account gets its own writable catalog, federated with a **shared, read-only catalog** for the whole machine. On a Windows install for everyone, the shared catalog lives in a machine-wide folder that every account can read, and publishing a folder into it takes Administrator approval — so what everyone sees is curated. On an install just for one user, both live in that user's profile, and "shared" means shared between the suite's applications rather than between people. The shared catalog starts with the sample San Francisco elevation dataset the suite ships.

Cost-Aware Routing

`find_optimal_for_operation` lets an agent name an operation and an area of interest and receive a ranked, costed, deduplicated list of catalog entries that can serve it — ordered by coverage and resolution, with access cost as the tiebreaker — and no hardcoded paths.

The five access tiers

Tier	Meaning	When a dataset gets it
LOCAL_HOT	Recently read by this process	Loaded within `cache.hot_ttl_seconds` (default 600 s)
LOCAL_COLD	On-disk local file; fast, but needs I/O	Local files not recently touched
REMOTE_DB	PostGIS — fast point lookup, costlier windowed reads	`postgis://`
REMOTE_COG	HTTP / S3 range-readable raster	`http(s)://`, `s3://`
REMOTE_WCS	OGC WCS and other on-demand services	`wcs://`, `arcgis://`, `wfs://`

> **Self-calibrating remote cost.** After the first read of a remote dataset, ForgeData stores the measured wall-clock latency and uses it instead of the static formula on later calls, for every remote tier. The measurement is persisted in the catalog, so it survives restarts, and `estimate_access_cost` reports whether an estimate is observed or heuristic.

How candidates are gathered, then ranked

Routing runs in two phases through one comparator. **Phase 1** surfaces locally available derived products whose operation is exactly the one requested, ahead of raw data. **Phase 2** takes raw datasets in the operation's required category whose bbox intersects the area, after filtering on resolution ceiling, an optional hard fidelity floor, minimum band count, an optional vector feature-class hint and, since 1.2.4, an optional date range. Zero-coverage candidates are dropped, and water-mask layers are never ranked. Both phases sort by:

1. **Adequate coverage** — a dataset covering at least half as much of your bbox as the best-covering one is adequate, and adequate datasets rank ahead of the rest.
2. **Resolution**, ascending, among the adequate ones (then coverage) — finer data wins its part of the area. Among the rest, coverage comes first, so a small fine patch never outranks a tile that covers the whole area.
3. **Extent-fit**, descending — the footprint closest to the area size wins.
4. **Access tier** — the final tiebreaker only. Locality breaks an otherwise perfect tie; it never overrides coverage or resolution.

Since 1.2.6, two 30 m tiles that split an area rank ahead of one 90 m tile that covers all of it, and the 90 m tile is then dropped as redundant — the same finest-first choice the elevation queries make when they read. Rank 1 can therefore cover only part of the area, with the rest of the mosaic behind it; each candidate's `coverage_fraction` says how much.

`preferWindowedRead` is deliberately not a filter or a rank key — it only feeds the reported load-time estimate — so an SRTM tile, which cannot be window-read, still ranks for a windowed-preferring operation such as viewshed and simply reads the whole tile. After sorting, a coverage dedup drops any raster candidate that adds no still-uncovered part of the area; it is skipped for vector layers, where a shared extent means different content, and for per-band imagery. Candidates carry a dataset id rather than a path, resolved with `get_dataset`, and a `reader_available` flag says whether a reader exists for the format.

Worked example

Area: a 1° × 1° bbox that one SRTM tile fully covers. The catalog holds thirty-six Arizona SRTM tiles (one intersecting), an Arizona-wide Copernicus DEM as a remote COG at 30 m, and a locally computed slope COG for that exact bbox.

Query	Result order	Why
SLOPE	slope COG → SRTM tile	Phase 1 surfaces the local derived slope first. The raw survivors tie on coverage and resolution, so extent-fit ranks the 1° tile ahead of the state-wide DEM; the DEM then adds no uncovered area, and the dedup drops it. Tier is never consulted
VIEWSHED	SRTM tile	No derived viewshed exists; the slope COG, being derived, is excluded from Phase 2. The SRTM tile stays valid despite the windowed-read preference and covers the area alone

Coverage fraction is always *your* bbox covered by the dataset. A fraction below 1.0 means several datasets are needed; the ranker handles that and the caller unions the top results.

When routing finds nothing

An empty result is not a dead end. It carries an `empty_reason` — including `no_temporal_match` since 1.2.4 — and, for coverage gaps, `acquisition_suggestions`: enabled, registered sources whose sync could acquire the area, each naming the next tool call. A `bootstrap_hint` names a keyless source type — Terrarium for elevation, Overpass for vector — when nothing registered can help.

The Operation Registry

Routing is driven by a registry of spatial operations grouped by what they need. Each declares its required category, a recommended resolution ceiling, whether windowed reads are preferred, and any minimum band count.

Group	Required category	Notable requirements
Elevation	ELEVATION	Slope, aspect, viewshed, roughness and TPI cap at 30 m; point, region, profile and hillshade uncapped
Hydrology	ELEVATION	Prefer windowed reads; most cap at 30 m; sink fill uncapped
Spectral indices	IMAGERY	Minimum band counts of two to four; BSI requires 10 m and four bands
Focal statistics	ELEVATION	Windowed-preferred; no resolution cap
Zonal statistics (1.2.4)	ELEVATION	Statistics inside a polygon
Interpolation	VECTOR	Point inputs for IDW, kriging and spline
Bathymetry	BATHYMETRY	Profile, water depth, dredge volume
Ocean acoustics (1.2.0)	the sound-velocity profile category	A sound-velocity profile input
Vector	VECTOR	Includes ForgeMind's inline-geometry buffer
Imagery	IMAGERY	Imagery classification
LiDAR	LIDAR	Windowed preference stays off until remote range reads land

An operation's **output category** is what the product is, not what it consumes: slope consumes ELEVATION but produces ANALYTICAL; hillshade is VISUALIZATION; sink fill returns a corrected DEM and stays ELEVATION. That mapping is what derived-product registration stamps automatically, so later routing finds products correctly.

Heterogeneous Sources, One Interface

Fourteen source-type implementations cover the common public and private geospatial sources. The set is fixed at compile time, deliberately; adding a type is one class, a validator, a schema and CLI wiring.

Source type	What it ingests	Auth
LOCAL_DIR	A directory of rasters (GeoTIFF, HGT, NetCDF, JPEG2000) and GeoJSON; optional file-watch pickup	None
STAC	SpatioTemporal Asset Catalog collections (e.g. Sentinel-2 on S3)	Optional bearer token
WCS	OGC Web Coverage Service — on-demand windowed raster	None
WFS	OGC Web Feature Service — GeoJSON output required at read time	Optional token
POSTGIS	PostGIS vector and raster, with server-side spatial filtering	Password variable
ARCGIS_REST	Esri FeatureServer and ImageServer layers	Optional token
OVERPASS	OpenStreetMap features via the Overpass API (tile grid)	None
OPENTOPO_API	OpenTopography elevation (tile grid; daily rate limits)	API key variable
USGS_3DEP	USGS 3DEP 10 m elevation over the United States (tile grid)	None
NOAA_API	NOAA datasets (tile grid)	None
COPERNICUS_CDS	Copernicus Climate Data Store (single request, hash-named output)	API key variable
TERRARIUM	Mapzen Terrarium terrain tiles — the keyless elevation bootstrap	None
TIGER_ROADS	TIGER/Line All Roads — named-road lookup and bbox queries	None
TIGER_ADDRFEAT	TIGER/Line address-range edges — address geocoding	None

Preview before you sync. `preview_data_source` returns the exact rows a sync would insert without touching the catalog or downloading bytes, each flagged if already catalogued. Catalog-style sources enumerate what they hold; tile-grid sources compute the tile list with no HTTP; single-request sources synthesise the file name the sync would write.

Sync and auto-sync. Sync runs on demand from the CLI or MCP, or on a schedule. A local directory with an interval also gets a file watcher that picks up dropped files and prunes deleted ones. One process per catalog owns the scheduler, enforced by an advisory lock on local disk.

The MCP Tool Surface

ForgeData's MCP server speaks stdio JSON-RPC and returns JSON. On the wire, local and `managed://` paths are externalised to absolute filesystem paths so external readers such as ForgeGIS can open them; remote URIs pass

through unchanged. The tools fall into these groups:

Group	Representative tools
Catalog queries	`list_datasets`, `get_dataset`, `get_status`, `search_by_bbox` / `_category` / `_tag` / `_source`
Catalog management	`insert_dataset`, `patch_dataset`, `delete_dataset` (optionally removing managed bytes), `gc_managed_storage`
Elevation	`elevation_at_point`, `elevation_at_points`, `elevation_in_region`, `elevation_profile` (multi-vertex), `elevation_stats_in_polygon`
Raster windows and maps	`read_raster_window` (the file's own data type, since contract 2.0.0), `render_map`
LiDAR	`lidar_elevation_at_point`, `lidar_density_in_region`, `lidar_to_raster`
Vector and location	`vector_features_in_bbox`, `lookup_road_by_name`, `roads_in_region`, `roads_near_point`, `geocode_address`
Routing	`find_optimal_for_operation`, `estimate_access_cost`
Data sources and sync	add, update, remove, preview and sync sources; sync status and cancel; source-type discovery and schemas
Derived products, publishing, scenarios	`register_derived_product`, `find_derived_products`, `probe_file`, `publish_file`, `probe_vector`, `publish_vector`, `find_by_content_hash`, `register_scenario`
Recordings and export	`index_recording`, `get_recording_manifest`, `export_stac`
Presence	`verify_dataset_presence`

Bounded replies (1.2.5). `list_datasets` and the `search_by_*` tools return **25 rows by default** (ceiling 500), with `total`, `truncated` and `next_offset`; `limit: 0` returns everything. Road and geocode match lists default to 100, 50 or 10 and are capped at 250. The geometry-returning tools stop inlining above **10 000 vertices** and say so in the reply; a caller that passes an `output_path` gets the full result written there, confined to the export root (`..` refused, symlinks re-checked). The Java API stays unbounded.

Structured, classified errors. Tool failures return `{ "error_code": "...", "message": "..." }` through a shared classifier: `source_file_missing`, `rate_limited`, `timeout`, `bbox_invalid`, `aoi_out_of_coverage`, `operation_not_supported`, `invalid_argument`, `database_error`, `forgegis_error`, `unknown_error`, and since 1.2.5 `catalog_read_only`, checked first. An empty answer is never silent: since 1.2.6 every reply that has no data says why in an `empty_reason` — a coverage gap (`no_aoi_coverage`), a covered area with no valid cell (`covered_no_cells`), an unreadable file — and every refusal carries an `error_code`; fields are only added, so existing consumers keep working. Tools layer their own codes on top — `read_raster_window` adds `windowed_read_unsupported` and `bbox_outside_dataset` among others; `elevation_in_region` returns `aoi_too_large` with the requested and maximum areas (250 000 km² by default); 1.2.6 adds `vector_crs_not_lonlat` for a local vector file that is not in longitude/latitude and `point_cloud_units_unknown` for a LiDAR cloud whose units cannot be known.

A versioned contract (1.2.6). The tool surface — names, input schemas, the units of every numeric parameter and reply value, and the closed vocabularies of error codes and empty reasons — is recorded in a generated contract snapshot that ships inside the jar and is reported by a hidden `contract_info` tool; a build that changes it without a version bump fails. 1.2.6 ships **contract 2.0.0**. Its one breaking change: `read_raster_window` now returns the file's real data type on every read — an INT16 DEM extract stays INT16 — where a single-strip or whole-image window used to come back as 32-bit float. Dimensions, CRS and NoData are unchanged. Consumers that pin the contract refresh their pin.

Reading Data

Path	Tools	What it does
Elevation	`elevation_at_point(s)`, `_in_region`, `_profile`, `_stats_in_polygon`	Point, many-point, region statistics and profiles through the cost model; `surface="bathymetry"` for sea-floor depth, `surface="auto"` falling back to bathymetry offshore. Since 1.2.6 each answer names the surface that answered and carries `vertical_units`, `vertical_datum` and `vertical_source` (from the file, or the documented product values for SRTM, Copernicus and 3DEP; `unknown` rather than guessed). Values are reported as stored, never converted, so a DEM in feet answers in feet and says so. Region statistics carry `coverage_fraction`; profile distances are WGS84 geodesics
Raster windows	`read_raster_window`	An area sub-window extracted to a local, georeferenced GeoTIFF in the file's own data type (contract 2.0.0) and, for a projected file, its own CRS; masks and per-band NoData are kept; extracts cached on disk
Vector	`vector_features_in_bbox`, road and geocode tools	One GeoJSON FeatureCollection, dispatched by scheme — PostGIS, ArcGIS, WFS, local GeoJSON or shapefile — with truncation reported; roads by name (TIGER, and OSM `name`/`highway` since 1.2.5); TIGER address geocoding
LiDAR	`lidar_*` tools	Ground elevation and point density over `.las`/`.laz`, local COPC reads, and rasterisation to DSM, DTM, intensity or density. Since 1.2.6 a cloud's units come from the file — metres, international and US survey feet are converted, compound-CRS 3DEP tiles read — and a cloud whose units cannot be known is refused
Maps	`render_map`	A hillshade image with a GeoJSON overlay, for a quick look

> **Honest water, from one seam.** DEMs store open water as a value, not an absence: SRTM flattens the ocean to exactly 0 m, which reads as solid ground. When the catalog holds a water-mask-tagged vector layer, ForgeData recodes over-water cells to NoData at its elevation read seam, before sampling, so point, region, profile and polygon elevation answers over a coastline are honest whichever source they come from — SRTM, Copernicus, 3DEP, Terrarium or remote WCS. It is a strict no-op until a water layer is present; genuine sea-level land is untouched, and bathymetry queries bypass the mask. Raster windows and rendered maps are served as stored.

Correctness fixes in 1.2.6. A projected DEM — a UTM publish, a LiDAR-derived elevation product, a WCS coverage in its native CRS — used to be read as if its metres were degrees, so a point inside it reported no coverage; the elevation

seam, raster windows, `render_map` and remote COG reads now work in the file's own CRS. Every distance, length and area ForgeData reports is measured on the WGS84 ellipsoid through ForgeGIS's measurement core, so it agrees with ForgeGIS Studio's Measure of the same line. Derived products no longer answer elevation queries, so a registered focal-statistics raster cannot be returned as metres of ground. The tile-gated integration tests and a set of GDAL-written fixtures are checked against GDAL's own values.

Derived Products and Publishing

Derived-product memory

A derived product is a dataset computed from another — slope from elevation, NDVI from imagery, a watershed. Registering one records `is_derived`, a `derived_op`, opaque `derived_params` and, when the parent is in the same catalog, a `parent_id` (ForgeMind carries a parent from another catalog in the parameters); the product is registered by reference, with an advisory warning if its path lies outside the read roots. Since 1.2.6 a local raster's CRS, extent and band count are read from the file, and a raster whose CRS cannot be identified is refused unless the caller names it, rather than being catalogued as EPSG:4326. Future routing prefers the pre-computed, locally available product over recomputing. An unrecognised operation still registers, stamped ANALYTICAL with a warning, so a new ForgeMind operation never blocks a layer. The reproducibility anchors are the parent (combined with the parent's content hash where one exists), the operation and its parameters.

Scenarios by reference (1.2.5)

`register_scenario` records a Seaglass Globe scenario in the catalog **by reference** — keyed on its library and scenario id, upserting and preserving fields it is not given — under a SCENARIO category, beside the data it sits on. Scenarios are catalogued, not routed.

The publish flow

`publish_file` contributes a local raster in one call: it probes georeferencing, copies the file into hash-sharded managed storage (optionally re-encoding it as a tiled COG with overviews), computes SHA-256 and inserts a catalog row with provenance. It is idempotent — a duplicate content hash returns the existing entry. `publish_vector` is the GeoJSON sibling and stores bytes unchanged; `probe_file` is an optional, side-effect-free pre-flight.

> **Atomic in seven steps.** Hash → in-process hash lock → check for an existing row → probe and apply hints → copy to a temp file and rename into `managed://ab/cd/{hash}.{ext}` → insert the row → release the lock. The partial unique index on the content hash catches any race that slips past the lock, even across two JVMs, and SQLite makes one insert lose cleanly.

Running ForgeData

Runtime. The server and command line run from one shaded jar, `forgedata-1.2.6-all.jar`, targeting Java 17 — no native install step (it carries its own per-platform SQLite driver), no daemon, no migrations to run by hand except for a shared read-only catalog, which `init-catalog` prepares. Since 1.2.6 the main Maven artifact, `forgedata-1.2.6.jar`, is a thin library jar whose dependencies Maven resolves, for teams embedding ForgeData; it no longer runs on its own. The suite runs it on a private Temurin JRE 25. The ForgeGIS GPU path is optional and falls back to the CPU. It logs to stderr so the JSON-RPC stream stays clean. A `forgedata.yml` (global and per bundle) tunes caches, timeouts, storage and extract limits; nearly every key has an environment-variable override, with precedence environment → bundle → global → default.

> **Paths that travel.** Since 1.2.6 a relative `download.dir` is anchored at the catalog's data root (`data.root`, else the catalog's own folder) rather than the JVM's working directory, so a sync run from anywhere writes tiles where the catalog's readers will find them; extracts, renders and the default export root follow. A bundle's relative `data.root` is anchored to the bundle, which is what makes relocatable data packs work. `FORGEDATA_DOWNLOAD_DIR` still overrides it with an absolute path (the Control Center sets one).

Catalog bundles. A bundle is a `catalog.gpkg`, the data it references and an optional `forgedata.yml`; it ships as an archive a recipient drops in and queries.

Health checks. `verify_dataset_presence` stat-checks local rows and stamps a verification time, optionally re-hashing files to detect corruption or replacement. The CLI `doctor` is diagnostic only — it never moves or deletes anything — and reports orphan directories and orphan rows, exiting non-zero on orphan rows. `gc_managed_storage` reclaims managed bytes no row references; by-reference rows are never collected.

Command line. Beyond sources, sync, road and geocode lookups and `doctor`, the CLI can list datasets, sample elevation, run routing, report status, export STAC and initialise a catalog.

Where a Routing Layer Earns Its Place

Tool	Solves	Leaves to the integrator
PostGIS	Stores and processes spatial data	Routing across sources that are not PostGIS
STAC catalogs	Index assets and metadata	Knowing what is warm or what an operation needs
GeoServer	Hosts OGC services	Decision-making for an agent; it serves maps to clients
Raw cloud COGs	Cheap storage at rest	Any discovery or ranking layer at all

Adopting any of these alone means hand-rolling the catalog, the operation-to-requirement binding, the fidelity-first ranking and the derived-product memory. ForgeData is that layer, built once.

Known Limits in 1.2.6

- **Formats without a reader**: EPT and remote COPC, PMTiles, FlatGeobuf and MVT are catalogued but flagged `reader_available: false`. WFS servers must offer GeoJSON output. Local shapefiles and GeoJSON must be in longitude/latitude; a projected one is refused (`vector_crs_not_lonlat`) rather than read wrong. A STAC source needs an `http(s)://` STAC API; static `catalog.json` catalogs are not read yet.
- **Extract cache**: a remote file changed on its server while its catalog row stays the same is not detected by the `read_raster_window` cache.
- **Roads**: OSM extracts that tag roads with `fclass` rather than `highway` (Geofabrik style) are not found by name yet.
- **Paging gaps**: `find_derived_products`, `list_data_sources` and `list_sync_jobs` are not paged; `vector_features_in_bbox`'s inline cap counts vertices, not bytes, so a point layer can still return many attribute-heavy features.
- **Water masking** needs a water-mask layer the customer supplies.
- **Concurrency**: one writer at a time; WAL and the auto-sync lock need local disk, not NFS or SMB.
- **Regions**: `elevation_in_region` is capped at 250 000 km² by default.

Specifications at a Glance

Catalog	One GeoPackage (`.gpkg`, SQLite) — `datasets` and `data_sources` tables; readable in QGIS and DB Browser
Federation	Catalog sets: reads fan out and merge with namespaced IDs, writes go to the primary; read-only secondaries (1.2.5)
Interfaces	MCP over stdio JSON-RPC · a command line · an embeddable Java library
Source types	Fourteen, fixed at compile time — LOCAL_DIR, STAC, WCS, WFS, POSTGIS, ARCGIS_REST, OVERPASS, OPENTOPO_API, USGS_3DEP, NOAA_API, COPERNICUS_CDS, TERRARIUM, TIGER_ROADS, TIGER_ADDRFEAT
Routing	Adequate coverage → finest resolution → extent-fit → access tier; derived products first; multi-resolution dedup for rasters; date filters
Access tiers	LOCAL_HOT, LOCAL_COLD, REMOTE_DB, REMOTE_COG, REMOTE_WCS — remote costs self-calibrate on measured latency
Concurrency	WAL (default) or DELETE journal; concurrent readers, one writer, busy-timeout waits; local disk

Replies	Dataset list and search page at 25 rows by default; geometry inlined up to 10 000 vertices, full results to a caller-supplied, sandboxed `output_path`; every no-data reply names an `empty_reason`; elevation answers carry vertical units, datum and source
MCP contract	Generated snapshot shipped in the jar, reported by `contract_info`; 2.0.0 at 1.2.6 — `read_raster_window` returns the file's real data type
Measurement	Distances, lengths and areas on the WGS84 ellipsoid (geodesics, via ForgeGIS)
Runtime	Runnable `forgedata-1.2.6-all.jar`, Java 17+; thin `forgedata-1.2.6.jar` for library use; no native install, no daemon; stderr logging
Validation	1,938 unit tests at commit 69fce61 (1,933 pass, 5 skip), no network; core queries checked against GDAL; opt-in remote integration tests
In the suite	MCP child of ForgeMind and of Seaglass Globe; embedded in ForgeGIS Studio; all read the same catalogs
License	Proprietary. Bundled with the commercially licensed Forge suite, not sold separately; evaluation builds available. Third-party components under their own licenses, including GeoTools (LGPL-2.1) and laszip4j (LGPL-3.0-or-later)

Licensing and Contact

ForgeData is proprietary software of Seaglass Foundry LLC — no license is granted except under a separate written agreement — distributed as part of the Forge suite through the Seaglass Control Center and not sold separately. The Forge suite is commercially licensed; terms are provided on request, and evaluation builds are available to qualified teams. Nothing in this brief is a grant of rights. The full reference set — MCP tool specifications, the `forgedata.yml` schema, environment variables, catalog DDL, ingester schemas, the operation registry, dataset-field semantics and the error taxonomy — ships with the suite documentation.

Contact. rich@seaglassfoundry.com · seaglassfoundry.com

ForgeData™, ForgeGIS™, ForgeMind™, ForgeGIS Studio™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. GeoPackage, SQLite, PostGIS, STAC, Sentinel-2, OGC, Esri, ArcGIS, OpenStreetMap, Overpass, OpenTopography, USGS, NOAA, Copernicus, Mapzen, TIGER/Line, AWS, Amazon S3, QGIS, GDAL, GeoServer, Java, Temurin, and the Model Context Protocol are marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.